Pandas

In [24]:
```
# Pandas is important Data Analysis tool kit
# Its a package
#Using PANDAS will find out missing data if they found in project
#Insertion and Deletion of rows and columns in table is done
#Automatic Data Alignment
# group by function on data
#slicing ,indexing and subsetting are used in PANDAS
#can work with multiple tables data with unions joins
#reshaping and reading any files using pandas
```

In [45]:
```
import pandas as pd
pd.__version__
```

Out[45]: '2.3.3'

In [46]:
```
df = pd.read_excel(r'C:\Users\venka\Downloads\1st, 2nd - Pandas\1st, 2nd - Pandas\C
```

In [47]:
```
df
#where df is data frame variable name
```

Out[47]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [48]:
```
len(df)
```

Out[48]: 195

In [49]:
```
df.shape  #shape will tell u about rows and columns of the table
```

Out[49]:
```
(195, 5)
```

In [50]:
```
df.columns  #its giving all column names
```

Out[50]:
```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
```

In [51]:
```
len(df.columns)
```

Out[51]:
```
5
```

In [52]:
```
df
```

Out[52]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [53]:
```
df.isnull()
#it is used to see if there are any null values in the table
# we get the data as false
```

Out[53]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
190	False	False	False	False	False
191	False	False	False	False	False
192	False	False	False	False	False
193	False	False	False	False	False
194	False	False	False	False	False

195 rows × 5 columns

In [54]:
```
df.isna()  #we can also isna instead of Isnull
```

Out[54]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
190	False	False	False	False	False
191	False	False	False	False	False
192	False	False	False	False	False
193	False	False	False	False	False
194	False	False	False	False	False

195 rows × 5 columns

In [55]:
```
df.isnull().sum()  #it gives sum of all null values showing how there are no null v
```

Out[55]:
```
CountryName      0
CountryCode      0
BirthRate        0
InternetUsers    0
IncomeGroup      0
dtype: int64
```

In [56]:
```
df.info()  #to know the information of data frame related to the
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 195 entries, 0 to 194
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CountryName    195 non-null    object
 1   CountryCode    195 non-null    object
 2   BirthRate      195 non-null    float64
 3   InternetUsers  195 non-null    float64
 4   IncomeGroup    195 non-null    object
dtypes: float64(2), object(3)
memory usage: 7.7+ KB
```

In [57]:
```
type(df)  #here it shows pandas data frame
# every categorical variblaes are considered by PANDAS as object
```

Out[57]:
```
pandas.core.frame.DataFrame
```

In [58]:
```
id(df)  #it gives address of the memory location
```

Out[58]:
```
2100117554576
```

In [61]:
```
df.dtypes  #it gives data types of all columns
```

Out[61]:
```
CountryName       object
CountryCode       object
BirthRate        float64
InternetUsers    float64
IncomeGroup       object
dtype: object
```

In [60]:
```
df.head()  #system bydefault prints top 5 rows from table
```

Out[60]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income

In [63]:
```
df.tail()  #it displays bottom 5 rows from the table records
```

Out[63]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

In [64]:
```
df.head(3) #we can add or delete the count for table rows to display
```

Out[64]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income

indexing and slicing in PANDAS

In [65]:
```
df[:]  #entire data frame is displayed
```

Out[65]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [66]:
```
df[5]   #It throws error in PANDAS but in numpy we dont get array
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_lo
c(self, key)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjec
tHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyObjec
tHashTable.get_item()

KeyError: 5

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[66], line 1
----> 1 df[5]   #It throws error in PANDAS but in numpy we dont get array

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4113, in DataFrame.__getitem
__(self, key)
   4111 if self.columns.nlevels > 1:
   4112     return self._getitem_multilevel(key)
-> 4113 indexer = self.columns.get_loc(key)
   4114 if is_integer(indexer):
   4115     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get_lo
c(self, key)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 5
```

In [67]:
```
df[5:10]  #it displayes records starting from 5th record to 9th record
```

Out[67]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
5	Argentina	ARG	17.716	59.9000	High income
6	Armenia	ARM	13.308	41.9000	Lower middle income
7	Antigua and Barbuda	ATG	16.447	63.4000	High income
8	Australia	AUS	13.200	83.0000	High income
9	Austria	AUT	9.400	80.6188	High income

In [68]:
```
df[3:]  #displays from 3rd to last
```

Out[68]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
5	Argentina	ARG	17.716	59.9	High income
6	Armenia	ARM	13.308	41.9	Lower middle income
7	Antigua and Barbuda	ATG	16.447	63.4	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

192 rows × 5 columns

In [69]:
```
df[:35] #from 1st record to 34 records
```

Out[69]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.90000	High income
1	Afghanistan	AFG	35.253	5.90000	Low income
2	Angola	AGO	45.985	19.10000	Upper middle income
3	Albania	ALB	12.877	57.20000	Upper middle income
4	United Arab Emirates	ARE	11.044	88.00000	High income
5	Argentina	ARG	17.716	59.90000	High income
6	Armenia	ARM	13.308	41.90000	Lower middle income
7	Antigua and Barbuda	ATG	16.447	63.40000	High income
8	Australia	AUS	13.200	83.00000	High income
9	Austria	AUT	9.400	80.61880	High income
10	Azerbaijan	AZE	18.300	58.70000	Upper middle income
11	Burundi	BDI	44.151	1.30000	Low income
12	Belgium	BEL	11.200	82.17020	High income
13	Benin	BEN	36.440	4.90000	Low income
14	Burkina Faso	BFA	40.551	9.10000	Low income
15	Bangladesh	BGD	20.142	6.63000	Lower middle income
16	Bulgaria	BGR	9.200	53.06150	Upper middle income
17	Bahrain	BHR	15.040	90.00004	High income
18	Bahamas, The	BHS	15.339	72.00000	High income
19	Bosnia and Herzegovina	BIH	9.062	57.79000	Upper middle income
20	Belarus	BLR	12.500	54.17000	Upper middle income
21	Belize	BLZ	23.092	33.60000	Upper middle income
22	Bermuda	BMU	10.400	95.30000	High income
23	Bolivia	BOL	24.236	36.94000	Lower middle income
24	Brazil	BRA	14.931	51.04000	Upper middle income
25	Barbados	BRB	12.188	73.00000	High income
26	Brunei Darussalam	BRN	16.405	64.50000	High income
27	Bhutan	BTN	18.134	29.90000	Lower middle income
28	Botswana	BWA	25.267	15.00000	Upper middle income
29	Central African Republic	CAF	34.076	3.50000	Low income

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
30	Canada	CAN	10.900	85.80000	High income
31	Switzerland	CHE	10.200	86.34000	High income
32	Chile	CHL	13.385	66.50000	High income
33	China	CHN	12.100	45.80000	Upper middle income
34	Cote d'Ivoire	CIV	37.320	8.40000	Lower middle income

In [73]:
```
df[0:200:25]  #it displays records from first record to 200 by counting every 25 re
```

Out[73]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.900000	High income
25	Barbados	BRB	12.188	73.000000	High income
50	Ecuador	ECU	21.070	40.353684	Upper middle income
75	Honduras	HND	21.593	17.800000	Lower middle income
100	Libya	LBY	21.425	16.500000	Upper middle income
125	Namibia	NAM	29.937	13.900000	Upper middle income
150	Sudan	SDN	33.477	22.700000	Lower middle income
175	Tunisia	TUN	19.800	43.800000	Upper middle income

In [74]:
```
df[::10] #it displays records from 1st to every 10th record
```

Out[74]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.900000	High income
10	Azerbaijan	AZE	18.300	58.700000	Upper middle income
20	Belarus	BLR	12.500	54.170000	Upper middle income
30	Canada	CAN	10.900	85.800000	High income
40	Costa Rica	CRI	15.022	45.960000	Upper middle income
50	Ecuador	ECU	21.070	40.353684	Upper middle income
60	Gabon	GAB	30.555	9.200000	Upper middle income
70	Greenland	GRL	14.500	65.800000	High income
80	India	IND	20.291	15.100000	Lower middle income
90	Kazakhstan	KAZ	22.730	54.000000	Upper middle income
100	Libya	LBY	21.425	16.500000	Upper middle income
110	Moldova	MDA	12.141	45.000000	Lower middle income
120	Mozambique	MOZ	39.705	5.400000	Low income
130	Netherlands	NLD	10.200	93.956400	High income
140	Poland	POL	9.600	62.849200	High income
150	Sudan	SDN	33.477	22.700000	Lower middle income
160	Suriname	SUR	18.455	37.400000	Upper middle income
170	Tajikistan	TJK	30.792	16.000000	Lower middle income
180	Uruguay	URY	14.374	57.690000	High income
190	Yemen, Rep.	YEM	32.947	20.000000	Lower middle income

In [76]:
```
df[::-50] #it displays the records from the end to start
```

Out[76]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
194	Zimbabwe	ZWE	35.715	18.5000	Low income
144	French Polynesia	PYF	16.393	56.8000	High income
94	Kiribati	KIR	29.044	11.5000	Lower middle income
44	Czech Republic	CZE	10.200	74.1104	High income

In [77]:
```
df['CountryName']
```

Out[77]:
```
0                     Aruba
1               Afghanistan
2                    Angola
3                   Albania
4      United Arab Emirates
                ...
190             Yemen, Rep.
191            South Africa
192        Congo, Dem. Rep.
193                  Zambia
194                Zimbabwe
Name: CountryName, Length: 195, dtype: object
```

In [78]:
```
df['CountryCode']
```

Out[78]:
```
0      ABW
1      AFG
2      AGO
3      ALB
4      ARE
      ...
190    YEM
191    ZAF
192    COD
193    ZMB
194    ZWE
Name: CountryCode, Length: 195, dtype: object
```

In [80]:
```
df[['BirthRate','InternetUsers']] # we need use double braces for multiple colms of
```

Out[80]:

	BirthRate	InternetUsers
0	10.244	78.9
1	35.253	5.9
2	45.985	19.1
3	12.877	57.2
4	11.044	88.0
...	...	...
190	32.947	20.0
191	20.850	46.5
192	42.394	2.2
193	40.471	15.4
194	35.715	18.5

195 rows × 2 columns

In [81]:
```
df.head(1)
```

Out[81]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income

In [82]:
```
df.columns
```

Out[82]:
```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
```

In [85]:
```
df_cat = df[['CountryName','CountryCode','IncomeGroup']]
df_cat.head(1)
```

Out[85]:

	CountryName	CountryCode	IncomeGroup
0	Aruba	ABW	High income

In [88]:
```
df_num = df[['BirthRate','InternetUsers']]
df_num.head(1)
```

Out[88]:

	BirthRate	InternetUsers
0	10.244	78.9

In [89]:
```
print(df.shape)
print(df_cat.shape)
print(df_num.shape)  #it gives u how many rows of data is there and how many column
```

```
(195, 5)
(195, 3)
(195, 2)
```

In [90]:
```
print(df.columns)
print()
print(df_cat.columns)
print()
print(df_num.columns)
```

```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
Index(['CountryName', 'CountryCode', 'IncomeGroup'], dtype='object')
Index(['BirthRate', 'InternetUsers'], dtype='object')
```

In [91]:
```
df.describe()  #this is used for descriptive statistcs
# eg..,mean, median, mode, stnd dev etc.,
#since it is related to stats it is used for numerical data only.
```

Out[91]:

	BirthRate	InternetUsers
count	195.000000	195.000000
mean	21.469928	42.076471
std	10.605467	29.030788
min	7.900000	0.900000
25%	12.120500	14.520000
50%	19.680000	41.000000
75%	29.759500	66.225000
max	49.661000	96.546800

In [92]:
```
df_num.describe()
```

Out[92]:

	BirthRate	InternetUsers
count	195.000000	195.000000
mean	21.469928	42.076471
std	10.605467	29.030788
min	7.900000	0.900000
25%	12.120500	14.520000
50%	19.680000	41.000000
75%	29.759500	66.225000
max	49.661000	96.546800

In [93]:
```
df_cat.describe()
```

Out[93]:

	CountryName	CountryCode	IncomeGroup
count	195	195	195
unique	195	195	4
top	Aruba	ABW	High income
freq	1	1	67

In [94]:
```
df
```

Out[94]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [96]:
```
df[df['InternetUsers'] <2]
```

Out[96]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
11	Burundi	BDI	44.151	1.3	Low income
52	Eritrea	ERI	34.800	0.9	Low income
55	Ethiopia	ETH	32.925	1.9	Low income
64	Guinea	GIN	37.337	1.6	Low income
117	Myanmar	MMR	18.119	1.6	Lower middle income
127	Niger	NER	49.661	1.7	Low income
154	Sierra Leone	SLE	36.729	1.7	Low income
156	Somalia	SOM	43.891	1.5	Low income
172	Timor-Leste	TLS	35.755	1.1	Lower middle income

In [99]:
```
df[df['BirthRate'] > 40]
```

Out[99]:

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
2	Angola	AGO	45.985	19.1	Upper middle income
11	Burundi	BDI	44.151	1.3	Low income
14	Burkina Faso	BFA	40.551	9.1	Low income
65	Gambia, The	GMB	42.525	14.0	Low income
115	Mali	MLI	44.138	3.5	Low income
127	Niger	NER	49.661	1.7	Low income
128	Nigeria	NGA	40.045	38.0	Lower middle income
156	Somalia	SOM	43.891	1.5	Low income
167	Chad	TCD	45.745	2.3	Low income
178	Uganda	UGA	43.474	16.2	Low income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income

In [101…

```
len(df[df['InternetUsers'] <2])
```

Out[101… 9

In [102…

```
df
```

Out[102…

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [104…
```
len(df[df['BirthRate'] > 40])
```

Out[104…
```
12
```

In [124…
```
df[(df.BirthRate > 40) & (df.InternetUsers < 2)]
```

Out[124…

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
11	Burundi	BDI	44.151	1.3	Low income
127	Niger	NER	49.661	1.7	Low income
156	Somalia	SOM	43.891	1.5	Low income

In [125…
```
df['IncomeGroup'] == 'High income'
```

Out[125…
```
0       True
1      False
2      False
3      False
4       True
       ...
190    False
191    False
192    False
193    False
194    False
Name: IncomeGroup, Length: 195, dtype: bool
```

In [126…
```
df[df['IncomeGroup'] == 'High income']
```

Out[126…

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.90	High income
4	United Arab Emirates	ARE	11.044	88.00	High income
5	Argentina	ARG	17.716	59.90	High income
7	Antigua and Barbuda	ATG	16.447	63.40	High income
8	Australia	AUS	13.200	83.00	High income
...	...	...	...	...	...
174	Trinidad and Tobago	TTO	14.590	63.80	High income
180	Uruguay	URY	14.374	57.69	High income
181	United States	USA	12.500	84.20	High income
184	Venezuela, RB	VEN	19.842	54.90	High income
185	Virgin Islands (U.S.)	VIR	10.700	45.30	High income

67 rows × 5 columns

In [127…

```
df[df['IncomeGroup'] == 'Low income']
```

Out[127…

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
1	Afghanistan	AFG	35.253	5.90	Low income
11	Burundi	BDI	44.151	1.30	Low income
13	Benin	BEN	36.440	4.90	Low income
14	Burkina Faso	BFA	40.551	9.10	Low income
29	Central African Republic	CAF	34.076	3.50	Low income
38	Comoros	COM	34.326	6.50	Low income
52	Eritrea	ERI	34.800	0.90	Low income
55	Ethiopia	ETH	32.925	1.90	Low income
64	Guinea	GIN	37.337	1.60	Low income
65	Gambia, The	GMB	42.525	14.00	Low income
66	Guinea-Bissau	GNB	37.503	3.10	Low income
77	Haiti	HTI	25.345	10.60	Low income
93	Cambodia	KHM	24.462	6.80	Low income
99	Liberia	LBR	35.521	3.20	Low income
111	Madagascar	MDG	34.686	3.00	Low income
115	Mali	MLI	44.138	3.50	Low income
120	Mozambique	MOZ	39.705	5.40	Low income
123	Malawi	MWI	39.459	5.05	Low income
127	Niger	NER	49.661	1.70	Low income
132	Nepal	NPL	20.923	13.30	Low income
148	Rwanda	RWA	32.689	9.00	Low income
154	Sierra Leone	SLE	36.729	1.70	Low income
156	Somalia	SOM	43.891	1.50	Low income
158	South Sudan	SSD	37.126	14.10	Low income
167	Chad	TCD	45.745	2.30	Low income
168	Togo	TGO	36.080	4.50	Low income
177	Tanzania	TZA	39.518	4.40	Low income
178	Uganda	UGA	43.474	16.20	Low income
192	Congo, Dem. Rep.	COD	42.394	2.20	Low income
194	Zimbabwe	ZWE	35.715	18.50	Low income

In [129…
```
df['IncomeGroup'].unique()  #ut gives all unique values
```

Out[129…
```
array(['High income', 'Low income', 'Upper middle income',
       'Lower middle income'], dtype=object)
```

In [130…
```
df['IncomeGroup'].nunique()  #it gives all no of unique values
```

Out[130…
```
4
```

In [ ]:
```
df['IncomeGroup'].nunique()
```

In [131…
```
df.columns
```

Out[131…
```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup'],
      dtype='object')
```

In [133…
```
df.columns = ['a','b','c','d','e']  #its a columnrename for pandas logic
df
```

Out[133…

	a	b	c	d	e
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income
2	Angola	AGO	45.985	19.1	Upper middle income
3	Albania	ALB	12.877	57.2	Upper middle income
4	United Arab Emirates	ARE	11.044	88.0	High income
...	...	...	...	...	...
190	Yemen, Rep.	YEM	32.947	20.0	Lower middle income
191	South Africa	ZAF	20.850	46.5	Upper middle income
192	Congo, Dem. Rep.	COD	42.394	2.2	Low income
193	Zambia	ZMB	40.471	15.4	Lower middle income
194	Zimbabwe	ZWE	35.715	18.5	Low income

195 rows × 5 columns

In [137…
```
df.columns = ['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup']
df.head(2)
```

Out[137…

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup
0	Aruba	ABW	10.244	78.9	High income
1	Afghanistan	AFG	35.253	5.9	Low income

In [140…

```
df['myCalc'] = df.BirthRate * df.InternetUsers
df.head()
```

Out[140…

	CountryName	CountryCode	BirthRate	InternetUsers	IncomeGroup	myCalc
0	Aruba	ABW	10.244	78.9	High income	808.2516
1	Afghanistan	AFG	35.253	5.9	Low income	207.9927
2	Angola	AGO	45.985	19.1	Upper middle income	878.3135
3	Albania	ALB	12.877	57.2	Upper middle income	736.5644
4	United Arab Emirates	ARE	11.044	88.0	High income	971.8720

In [142…

```
df.columns
```

Out[142…

```
Index(['CountryName', 'CountryCode', 'BirthRate', 'InternetUsers',
       'IncomeGroup', 'myCalc'],
      dtype='object')
```

In [147…

```
df=df.drop('myCalc',axis = 0)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[147], line 1
----> 1 df=df.drop(        ,axis = 0)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:5603, in DataFrame.drop(sel
f, labels, axis, index, columns, level, inplace, errors)
   5455 def drop(
   5456     self,
   5457     labels: IndexLabel | None = None,
   (...)   5464     errors: IgnoreRaise = "raise",
   5465 ) -> DataFrame | None:
   5466     """
   5467     Drop specified labels from rows or columns.
   5468
   (...)   5601             weight  1.0     0.8
   5602     """
-> 5603     return super().drop(
   5604         labels=labels,
   5605         axis=axis,
   5606         index=index,
   5607         columns=columns,
   5608         level=level,
   5609         inplace=inplace,
   5610         errors=errors,
   5611     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4810, in NDFrame.drop(sel
f, labels, axis, index, columns, level, inplace, errors)
   4808 for axis, labels in axes.items():
   4809     if labels is not None:
-> 4810         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4812 if inplace:
   4813     self._update_inplace(obj)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4852, in NDFrame._drop_axi
s(self, labels, axis, level, errors, only_slice)
   4850         new_axis = axis.drop(labels, level=level, errors=errors)
   4851     else:
-> 4852         new_axis = axis.drop(labels, errors=errors)
   4853     indexer = axis.get_indexer(new_axis)
   4855 # Case for non-unique axis
   4856 else:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:7136, in Index.drop(s
elf, labels, errors)
   7134 if mask.any():
   7135     if errors != "ignore":
-> 7136         raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7137     indexer = indexer[~mask]
   7138 return self.delete(indexer)

KeyError: "['myCalc'] not found in axis"
```

In [148…

```
df=df.drop('myCalc',axis = 1)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[148], line 1
----> 1 df=df.drop(        ,axis = 1)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:5603, in DataFrame.drop(sel
f, labels, axis, index, columns, level, inplace, errors)
   5455 def drop(
   5456     self,
   5457     labels: IndexLabel | None = None,
   (...)   5464     errors: IgnoreRaise = "raise",
   5465 ) -> DataFrame | None:
   5466     """
   5467     Drop specified labels from rows or columns.
   5468
   (...)   5601             weight  1.0     0.8
   5602     """
-> 5603     return super().drop(
   5604         labels=labels,
   5605         axis=axis,
   5606         index=index,
   5607         columns=columns,
   5608         level=level,
   5609         inplace=inplace,
   5610         errors=errors,
   5611     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4810, in NDFrame.drop(sel
f, labels, axis, index, columns, level, inplace, errors)
   4808 for axis, labels in axes.items():
   4809     if labels is not None:
-> 4810         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4812 if inplace:
   4813     self._update_inplace(obj)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4852, in NDFrame._drop_axi
s(self, labels, axis, level, errors, only_slice)
   4850         new_axis = axis.drop(labels, level=level, errors=errors)
   4851     else:
-> 4852         new_axis = axis.drop(labels, errors=errors)
   4853     indexer = axis.get_indexer(new_axis)
   4855 # Case for non-unique axis
   4856 else:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:7136, in Index.drop(s
elf, labels, errors)
   7134 if mask.any():
   7135     if errors != "ignore":
-> 7136         raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7137     indexer = indexer[~mask]
   7138 return self.delete(indexer)

KeyError: "['myCalc'] not found in axis"
```

SEABORN

In [157…

```
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
plt.rcParams['figure.figsize']= 6,2

# this is used to maintain size of the graphs created based on the data.

import warnings
warnings.filterwarnings('ignore') # os error
#figured warnings is used to ignore warnings which occur due to OS updates /softwar
```

In [153…

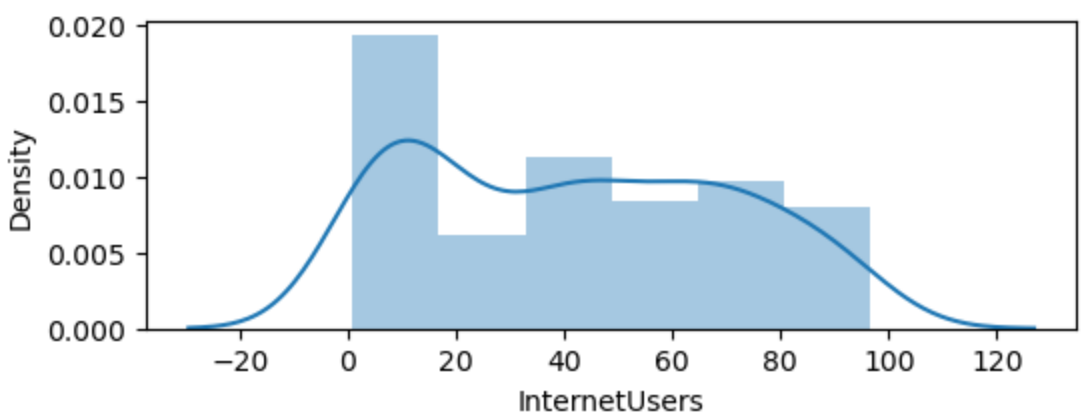
```
df['InternetUsers']
```

Out[153…

```
0      78.9
1       5.9
2      19.1
3      57.2
4      88.0
       ...
190    20.0
191    46.5
192     2.2
193    15.4
194    18.5
Name: InternetUsers, Length: 195, dtype: float64
```

In [159…

```
vis1=sns.distplot(df['InternetUsers'])
# distplot is advance graph function only used in sns
plt.show()   #to display the graph
```

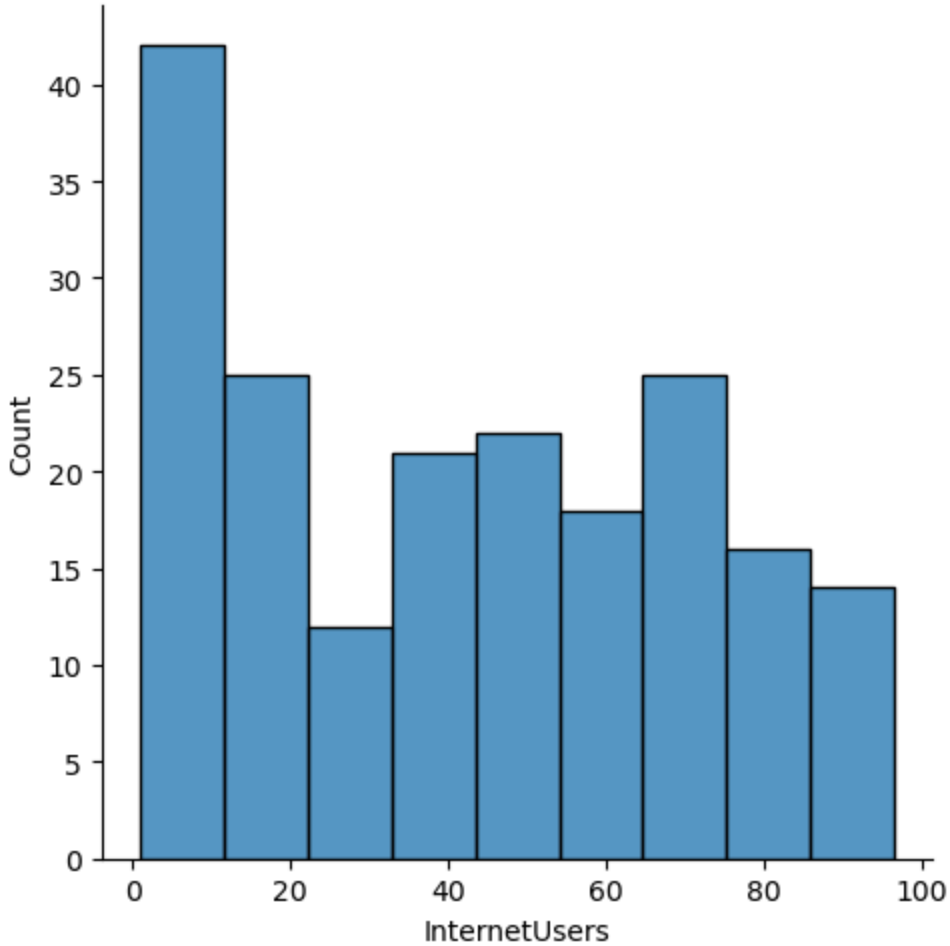


In [155…

```
%matplotlib inline
plt.rcParams['figure.figsize']= 10,2
plt.show()
```

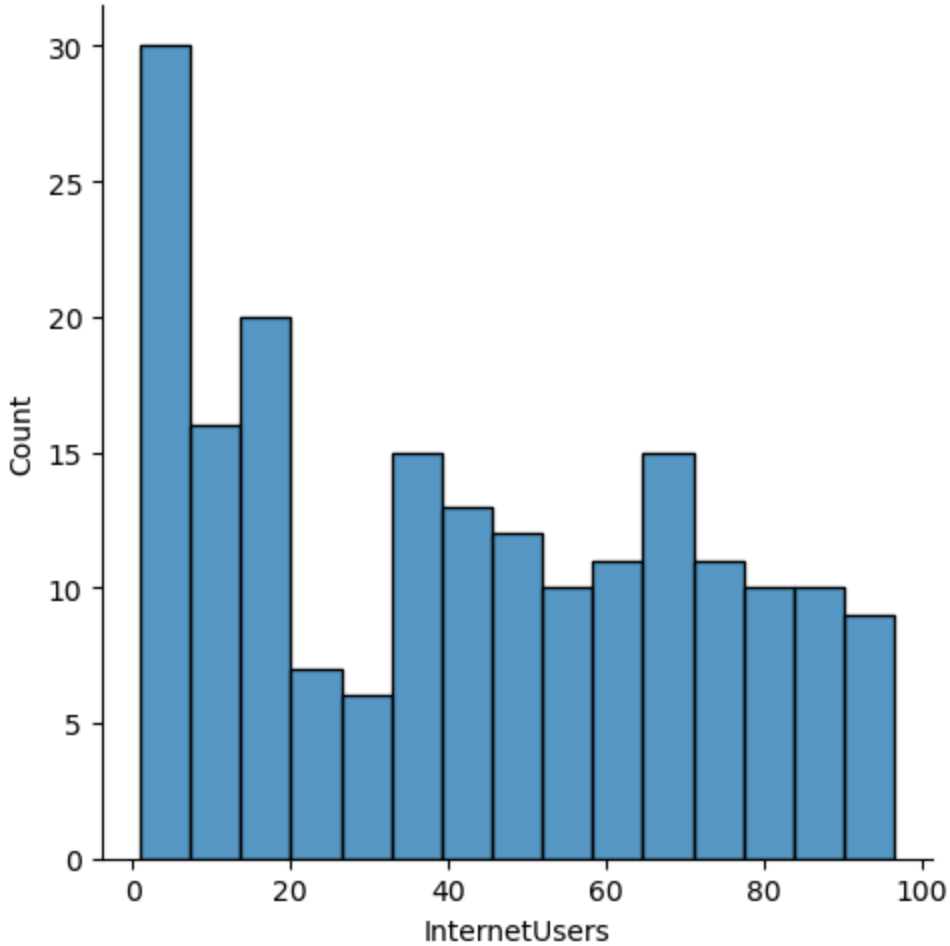
In [160…

```
vis2=sns.displot(df['InternetUsers'])
```



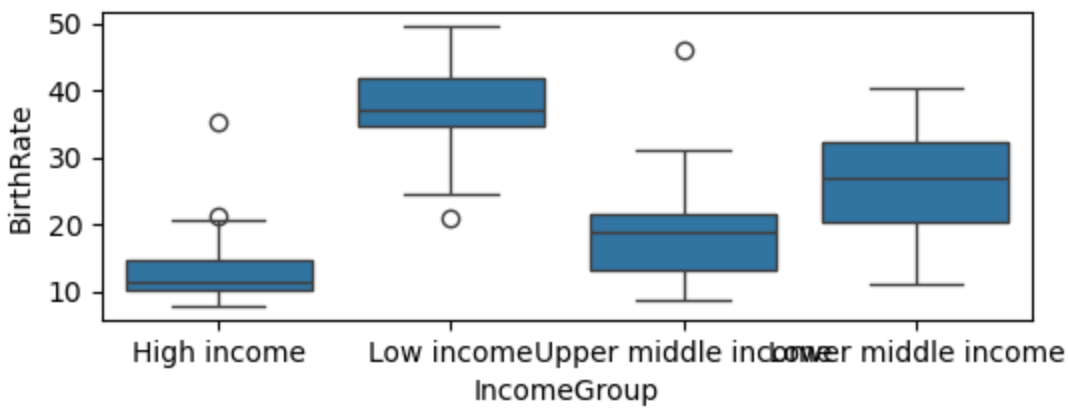
In [ ]:
```
# the missing curve is called as distributions line
```

In [161…
```
vis2=sns.displot(df['InternetUsers'] , bins=15)
# when u add bins it give little clear description of graph
```



In [163…

```
# BOX plot
vis4 = sns.boxplot(data = df,x= 'IncomeGroup', y='BirthRate') #Bi-Variate Analysis)
```

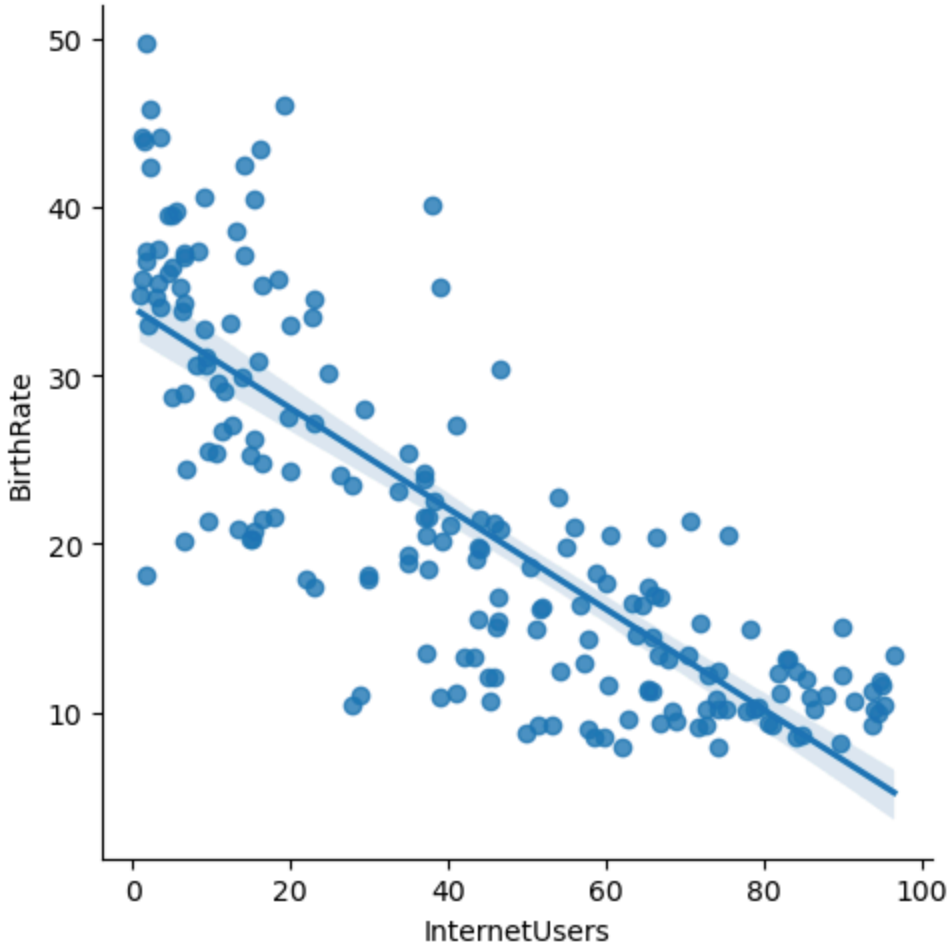


In [ ]:

```
# Outlier in statistics is a data point which is far from other observations points
# Its also called as anomaly detection
#these outlier impact machine learning model accuracy
# To Adjust these Outliers we introduce Probability Concept is being used.
# Theere are 2 algorithms used for Probability concept
# 1.logical regression and 2.naiver bayes
```

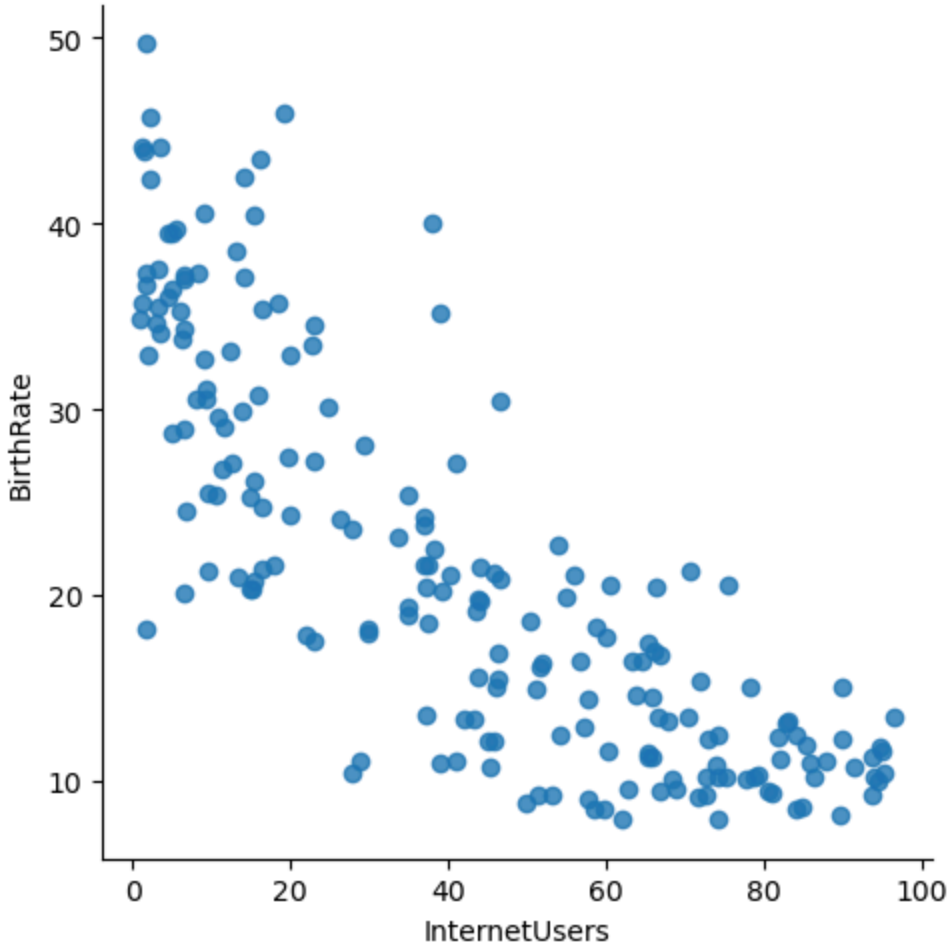
In [164…

```
vis5 = sns.lmplot(data = df,x= 'InternetUsers', y='BirthRate')
#lm plot has several dots and each dot is one data point of one record
```



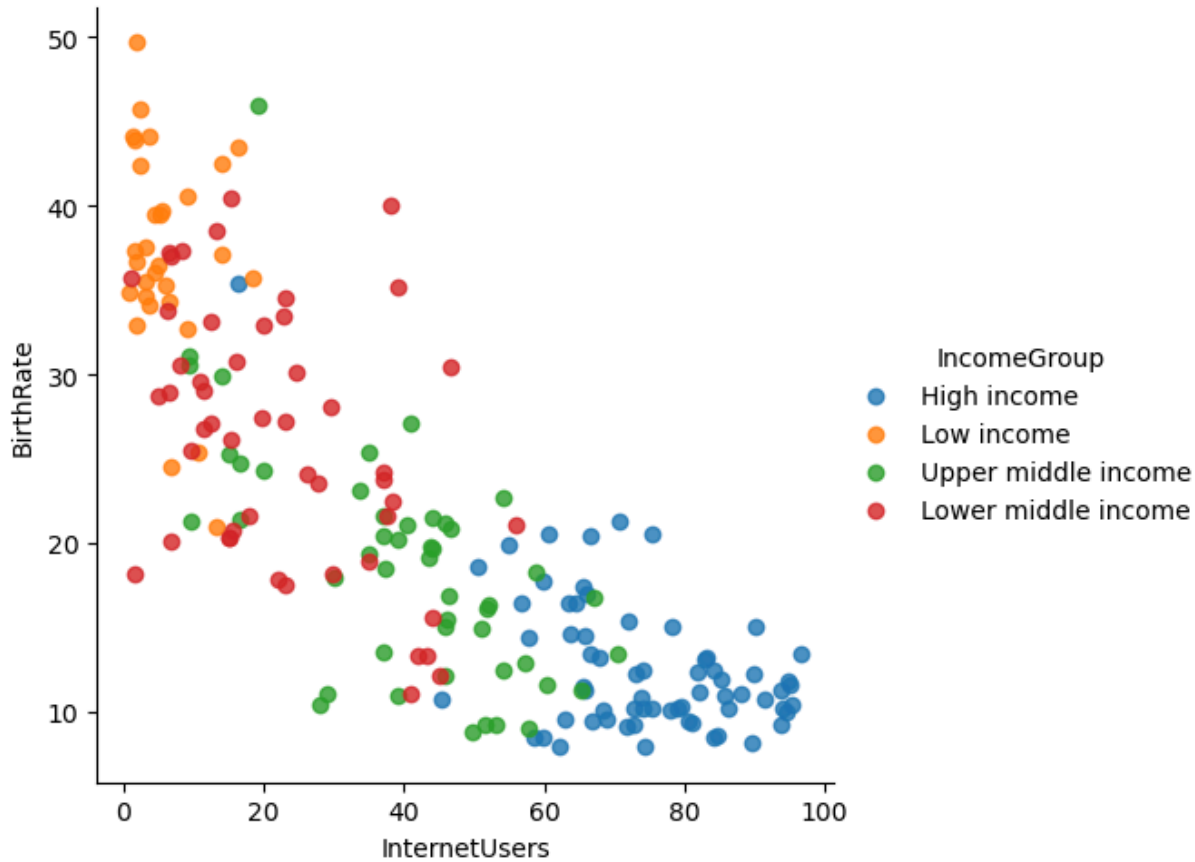
In [166…

```
vis6 = sns.lmplot(data = df,x= 'InternetUsers', y='BirthRate', fit_reg = False)
#fit_reg will remove the line from the graph
```



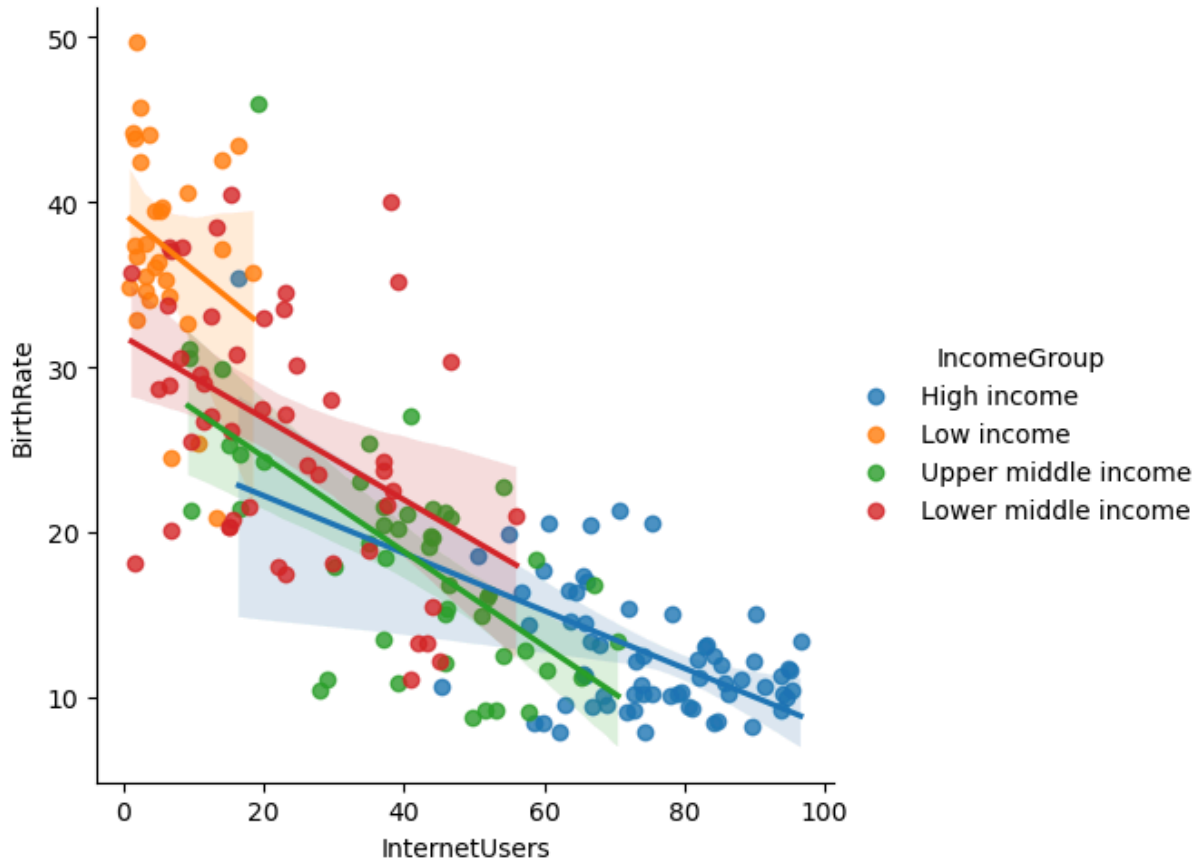
In [169…

```
vis7 = sns.lmplot(data = df,x= 'InternetUsers', y='BirthRate',
                  fit_reg = False, hue ='IncomeGroup')
#where hue is for colorparameter like legend in matplotlib
```



In [170…

```
vis7 = sns.lmplot(data = df,x= 'InternetUsers', y='BirthRate',
                  fit_reg = True, hue ='IncomeGroup')
#FIT_REG GIVES LINES OF THE DIFFERENCES
```


50
40
30
20
10
BirthRate
0
20
40
60
80
100
InternetUsers
IncomeGroup
High income
Low income
Upper middle income
Lower middle income

In [ ]:

In [ ]:

In [ ]:

In [ ]: